BioRescue

1. **Idea del producto y Product Goal**

Problema

En muchos centros de rescate e instituciones de protección de fauna silvestre, el registro de los animales rescatados y el seguimiento de su estado de salud aún se llevan de manera informal (en hojas de papel, planillas sueltas o archivos de texto sin estructura). Esto genera problemas graves como:

- Pérdida de información importante sobre los animales.
- Dificultad para saber rápidamente cuántos especímenes hay y en qué estado se encuentran.
- Falta de un historial claro (trazabilidad) del estado clínico de cada animal desde que ingresa hasta su liberación o traslado.

Solución: **BioRescue** es un programa de consola desarrollado en **Java** que centraliza y organiza toda esta información. Con esta herramienta, el personal del centro de rescate puede:

- Registrar la entrada de nuevos animales de forma estructurada.
- Consultar y buscar especímenes rápidamente por su código de ficha.
- Actualizar la condición de salud de cada animal a medida que evoluciona.
- Filtrar especímenes según la zona geográfica donde fueron rescatados.
- Guardar todo automáticamente en un archivo local (especimenes.csv) para que la información no se pierda al cerrar el programa.

Product Goal: Entregar una aplicación de consola ligera, estable y fácil de usar que permita al personal de un centro de rescate gestionar el ciclo completo de vida de los datos de la fauna silvestre (registro, consulta, edición, eliminación y filtrado), garantizando que la información quede guardada de forma segura y permanente en el equipo.

2. Marco de trabajo Scrum y organización del equipo

Marco de trabajo: Utilizamos la metodología **Scrum** para organizar el proyecto. Esto significa que no dejamos todo para el final, sino que entregamos versiones funcionales y probadas de **BioRescue** de forma periódica, ajustando el desarrollo a medida que avanzamos., apoyándose en los pilares empíricos de transparencia, inspección y adaptación.

Distribución de roles: Dado que somos un equipo de 2 personas, ambos participamos activamente en el diseño, programación y pruebas del software (*Developers*), pero dividimos los roles principales para mantener el orden:

- **Karen — Product Owner & Developer:** Se encarga de definir las funcionalidades del sistema, priorizar las tareas en el listado del proyecto (Product Backlog) y asegurar que el software cumpla con lo que se requiere.
- **Kevin — Scrum Master & Developer:** Se encarga de coordinar la entrega de tareas a tiempo, organizar las reuniones rápidas de avance y resolver cualquier problema o bloqueo técnico o metodológico que impida el progreso.

¿Cuándo consideramos que una tarea está terminada? (Definition of Done - DoD)

1. **Funciona:** El código corre sin errores y cumple con lo que se planificó.
2. **Está probado:** Se hicieron pruebas probando casos normales y casos con datos incorrectos o límite para evitar fallas.
3. **Está integrado:** El código se subió y unió a la rama principal del repositorio en GitHub sin romper lo que ya existía.
4. **Está documentado:** Si la funcionalidad requería manuales, diagramas o notas, estos quedaron actualizados.

3. Lista de tareas e historias de usuario (Product Backlog)

El *Product Backlog* de BioRescue reúne las funcionalidades principales requeridas por los operadores del centro de rescate para gestionar la información de los especímenes:

- **HU-01 — Registro de nuevos especímenes:** Como operador, quiero registrar la información básica de un espécimen (código, especie, estado de salud, zona de rescate) para ingresar animales al sistema de forma rápida.
- **HU-02 — Consulta e historial:** Como operador, quiero buscar y visualizar la lista completa de especímenes según su código de ficha para revisar sus datos en cualquier momento.
- **HU-03 — Actualización de estado de salud:** Como operador, quiero modificar el estado clínico de un espécimen para registrar su avance o cambios en su proceso de rehabilitación.
- **HU-04 — Depuración de registros (Eliminar):** Como operador, quiero eliminar registros ingresados por error para mantener la base de datos limpia y sin duplicados.
- **HU-05 — Filtro por zona geográfica:** Como operador, quiero filtrar el listado de animales según la zona de rescate para coordinar mejor la atención por territorios.
- **HU-06 — Manejo de errores y estabilidad:** Como operador, quiero que el sistema me avise cuando introduzco un dato inválido (como un texto en lugar de un número o una fecha incorrecta) sin que la aplicación se cierre o colapse.

4. Planificación y entregas (Cronología de Sprints)

El desarrollo del sistema se organizó en 3 ciclos de trabajo (Sprints) para entregar avances funcionales progresivamente:

Sprint 1: Base del sistema y registro básico

Objetivo: Definir la estructura de datos principal y permitir el ingreso de datos básicos.

- **Entregables:**
    - Estructura y modelo del objeto Especimen.
    - Lógica para guardar y consultar datos temporalmente en memoria.
    - Menú inicial con opciones de **Registrar** y **Listar** especímenes.

Sprint 2: Búsqueda, edición y eliminación

Objetivo: Completar el ciclo de gestión de datos (operaciones CRUD).

- **Entregables:**
    - Motor de búsqueda por código único de ficha.
    - Formulario/función para actualizar el estado clínico de los especímenes.
    - Opción para eliminar registros del sistema.

Sprint 3: Guardado en disco, filtros y pruebas de estabilidad

Objetivo: Asegurar que los datos no se pierdan al cerrar la aplicación y garantizar que la interfaz sea estable.

- **Entregables:**
    - Guardado y lectura automática de información desde un archivo de texto (especimenes.csv).
    - Módulo de filtrado rápido por zona de rescate.
    - Validaciones de entrada de datos (fechas, textos y números) para evitar cierres inesperados.
    - Pruebas de integración para validar la estabilidad general del sistema.